MONASH University

Monash University

ECE2072 Assignment Report

Processor in Verilog HDL

Loo Yi Ren Dillon (33455953)
10-16-2023

MONASH University

Contents

Introduction

This formal report presents process of designing and coding a processor using Verilog. Building a processor is a fundamental task in computer engineering, involving intricate work in digital logic design and control implementation. This report provides a detailed account of the project, outlining the decisions made, challenges encountered, and innovative solutions developed throughout the journey. The objective is to gain a deeper understanding of processor architecture and strengthen coding skills while contributing to the field of digital systems.

Components

Sign Extender

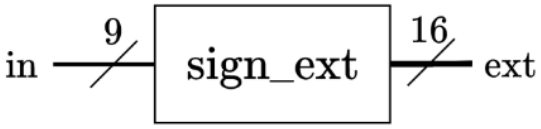

Figure 1: high level diagram of sign extender module.

The sign extender module serves the role of expanding the 9-bit input of the processor into a 16-bit binary representation, accommodating both positive and negative 2's complement numbers. It's operation involves analysing the most significant bit (MSB) of the input and replicating it six times to the left, thereby generating the extended 16-bit output.

Tick Counter

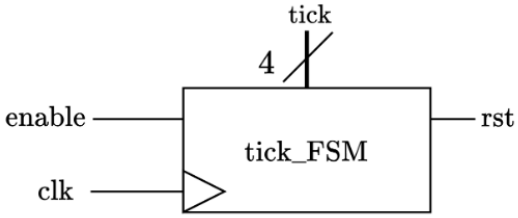

Figure 2: high level diagram of ALU module.

Table 1: encoding table for tick counter.

States	Encoding
state 1	0001
state 2	0010
state 3	0100
state 4	1000

The tick counter is a finite state machine that maintains the current state of the processor. It consists of 4 states following the one hot encoding to represent each state. It updates to next state accordingly during each clock if the enable pin is turned on. If reset pin is on, the tick counter will return to the 0^{th} state during the next positive edge of the clock.

ALU

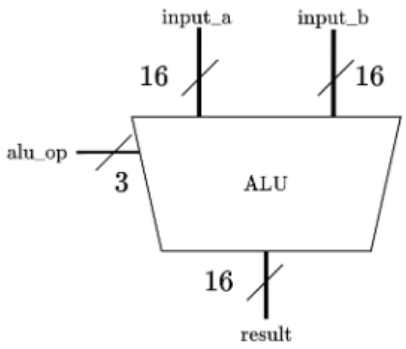

Figure 3: high level diagram of ALU module.

The arithmetic logic unit (ALU) is the component that performs arithmetic operations for the processor, including:

- Addition
- Subtraction
- Multiplication
- Bit-shifting

The ALU's requirement as per instructed is to take in two 16-bit values and perform the corresponding operation as shown above depending on alu_op's value, and result is wired to input of shift register G, subsequently output to bus which will be displayed on LEDs.

Register

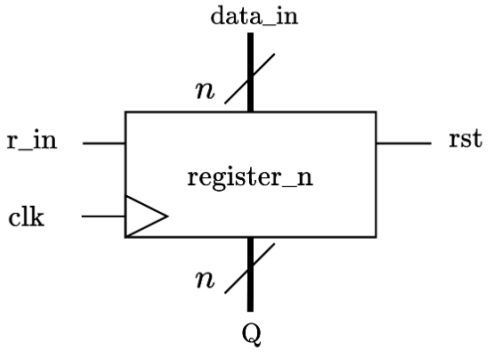

Figure 4: high level diagram of register module.

Registers in a processor are high-speed storage elements used to store and manage data and instructions during the execution of a computer program. The processor employs a variety of registers, such as 8 general-purpose registers for data storage, instruction registers for storing instructions, and address registers. The control unit governs the enabling of specific registers in different states for various operations, and when the reset pin is activated, the stored values are cleared.

Multiplexer

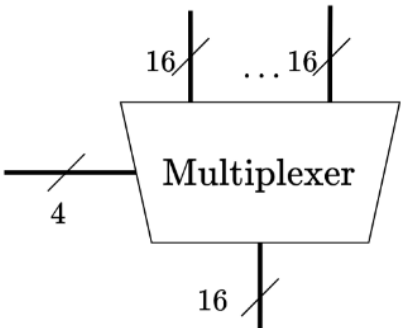

Figure 5: high level diagram of Multiplexer module

Table 2: encoding table for Multiplexer.

Output	Bus control (decimal)
Register 0	0
Register 1	1
Register 2	2
Register 3	3
Register 4	4
Register 5	5
Register 6	6
Register 7	7
Sign extend	8
Register G	9

The multiplexer is the component responsible of managing the selection of data to be transmitted onto the bus wire. The control pin is handled by the Control Unit, which its value will determine which of the 10 inputs of the multiplexer will appear on the output, which is connected to the bus wire.

Resource usage and timing analysis

Components	Logic Usage	Timing Constraints
Sign Extender	26 Logic Cells + 0 DSPs	1092.9 MHz
Tick Counter	17 Logic Cells + 0 DSPs	397.14 MHz
Multiplexer	53 Logic Cells + 0 DSPs	127.55 MHz
Register	51 Logic Cells + 0 DSPs	562.75 MHz
ALU	281 Logic Cells + 10 DSPs	52.82 MHz

Table 3: Logic usage and timing constraint data.

In summary, the Sign Extender boasts 26 logic cells and operates at an impressive 1092.9 MHz, making it ideal for real-time signal processing. The Tick Counter is compact with 17 logic cells, clocking in at 397.14 MHz, suitable for accurate event counting. The Multiplexer uses 53 logic cells, striking a balance at 127.55 MHz for data routing. The Register is equipped with 51 logic cells, efficiently managing data at 562.75 MHz. The ALU"is the most complex, requiring 281 logic cells, albeit operating at 52.82 MHz due to its complexity. These insights provide a valuable foundation for evaluating and optimizing system efficiency and performance.

Design & implemented Datapath

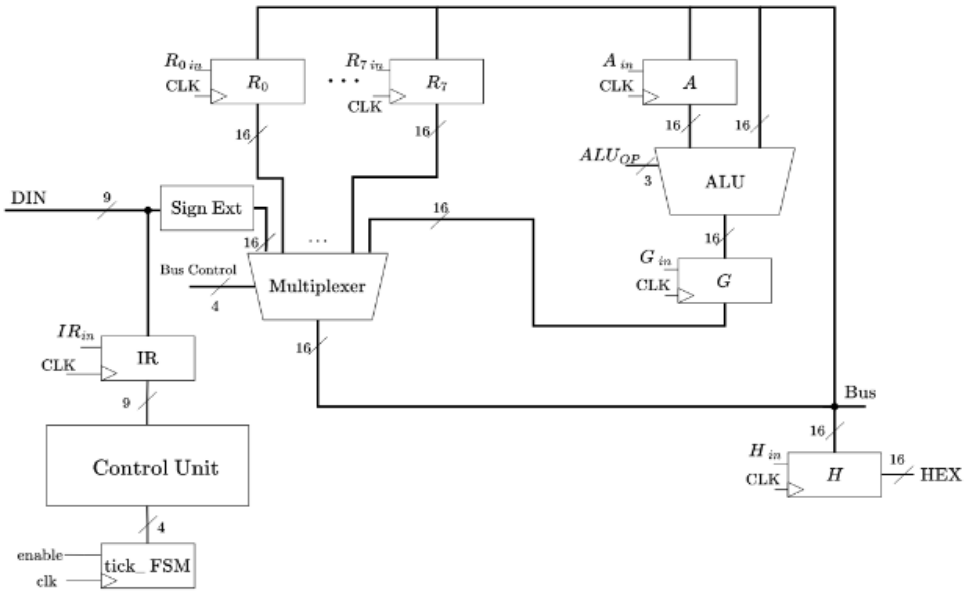

Figure 6: High-level topology of the processor.

The provided diagram illustrates the Datapath configuration employed within the processor. Every aspect of control, ranging from the regulation of Multiplexer bus selection to ALU operation, as well as the activation of individual registers, is centralized at the control unit. The processor initiates its operations by reading input data either during the initial state, which marks the processor's inception, or at the fourth state. The implementation of this Datapath shown above presents a few conditions:

1. Every single data pass through Multiplexer, except for value stored in register H which is connected to display output port of the processor.
2. Values to be loaded into the bus wire always comes from register IR, passing through the Sign Extender Module, extending any bit-length value to be extended.
3. Register A is responsible for latching the first value if operation involves 2 registers.
4. Register H holds the value that has to be outputted from the display output port of the processor.

. having different operations for different instructions, and categorised as below:

- Arithmetic (addition, subtraction, multiplication)
- Display (display on 7-segment)
- Bit-shifting (shift right, shift left)
- Move immediate

Arithmetic

The arithmetic operations for this processor are operations that functions through all four ticks, and 4 processor operations fall under this category.

Tick 1: The processor stores the 9-bit input in the instructions register, and it is split into 3 storages. Opcode for bit 8 to 6, Rx for bit 5 to 3, Ry for bit 2 to 0/immediate value depending on the instruction's opcode.

Tick 2: In this clock cycle, the control unit reads the opcode to determine the ALU operation selection. Simultaneously, if instruction is to add Rx & Ry, it loads the value of Rx into bus wire. Else if opcode is to add Rx and an immediate value, the 9-bit data input will be placed on to the bus instead. Afterwards, register A will be enabled, setting up the first operand for the ALU operation in the subsequent state.

Tick 3: This clock cycle starts with register A shifting the previously placed value on the bus, register A is then promptly deactivated. During this time, if opcode is to perform addition on Rx and Ry, value stored in register Ry will be placed onto the bus. Else if opcode is to add Rx value to immediate value. At this point, the ALU's output is ready but not yet stored, as it awaits the next state. To prepare for this, register G's enable pin is activated in this state.

Tick 4: In this clock cycle, register G promptly stores the value of the ALU's result output at the positive edge of the clock and is then deactivated simultaneously. The control pin of the multiplexer is set to value 9, allowing register G's value to be outputted onto the bus. The output of this operation will then be stored in register Rx.

Display

The display operation for this processor is a two-stage function that exclusively operates during ticks 1 and 2, remaining idle for the remaining clock cycles. In the second clock cycle (tick 2), the data stored in register H, which contains the address of a register, is transmitted through the display output port.

Tick 1: The value of 9-bit input is saved into instructions register, and it is split into 3 storages. Opcode for bit 8 to 6, Rx for bit 5 to 3, Ry for bit 2 to 0, but since only one register storage is needed, address Ry is redundant. If opcode is 3'b 000, register H's enable pin is turned on (prepare for next tick). At the same time, bus control allows value stored address Rx into bus wire, ready for register H to take data during the next tick.

Tick 2: Register H instantly takes value before it's disabled, outputting the value of data stored in register Rx through register H's output, which is connected to the display output port of the processor.

Tick 3: idle

Tick 4: idle

Bit-shifting

The bit-shifting operation is akin to arithmetic operations, with the key distinction of requiring one less clock cycle since only one value is needed for the ALU to perform the shift, in contrast to mathematical operations involving two values. This operation unfolds over three clock cycles, and it remains idle during the fourth.

Tick 1: The processor stores the 9-bit input in the instructions register, and it is split into 3 storages. Opcode for bit 8 to 6, Rx for bit 5 to 3, Ry for bit 2 to 0, and in this case Ry is redundant since only one register address is required. If opcode is 3'b 101 or 3'b 110, the ALU's operation control pin is set for 'Shift left' or 'Shift right,' respectively.

Tick 2: The value stored in register Rx is loaded onto the bus wire, while simultaneously enabling register G, preparing it for the next state, where it will store a value.

Tick 3: In this clock cycle, the shifted data value is instantly shifted into register G before it's deactivated. The bus control allows the value stored in register G to appear on the bus, allowing this value to be saved back into the same register.

Tick 4: idle

Move immediate

The move immediate operation's purpose is to store an immediate value from input data directly to the specific register Rx (register address from instruction register bit 5 to 3).

Tick 1: The processor stores the 9-bit input in the instructions register, and it is split into 3 storages. Opcode for bit 8 to 6, Rx for bit 5 to 3, Ry for bit 2 to 0, but in this case Ry is irrelevant. If 3'b 111 is read, the processor will load immediate value (data in) onto bus and turning on enable pin of register Rx.

Tick 2: Specified register instantly stores data before it gets disabled.

Tick 3: idle

Tick 4: idle

Product testing

Sign Extender testbench

In the sign extender testbench, the testing procedure involves increasing the input value one bit every 10 nanoseconds, while simultaneously recording the corresponding outputs. This output is subsequently compared to the input, focusing on the six replicated bits added to it, and their alignment with the most significant bit (MSB) of the input. When a disparity is detected between the replicated bits and the MSB, an error counter is incremented by 1 as part of the validation process. As shown in appendix, the error counter is 0 and hence the module is functioning as expected.

Tick Counter testbench

To test the tick counter, another simple approach is employed to control when the reset and enable pins are toggled on or off. This methodology offers an effective means to expose potential glitches, especially when hardware testing has already validated that the module operates as anticipated. The output tick is subsequently examined to verify whether it aligns with the expected conditions based on the activation or deactivation of the reset and enable pins. Furthermore, due to the module's simplicity, encompassing only four states, it lends itself to straightforward validation by inspecting the waveform. As illustrated in Appendix B, the visual representation demonstrates that the module operates without errors.

ALU testbench

To streamline the testing process for the ALU module, a simplified approach is employed due to its complexity. Given that the ALU is responsible for five core operations, the increment of the ALU Operation value is divided into five separate values, each executed simultaneously. This strategy eliminates the need to modify both alu operation port independently during testing. Consequently, all five operations' outputs are generated as simultaneously, simplifying the validation process. It's important to note that the testbench examines shorter bit lengths for both Input A and Input B. This approach aligns with the processor's specifications, involving either two 3-bit inputs or one 3-bit input combined with an immediate value that is up to 9 bits maximum. Therefore, both inputs are tested up to 9-bits. The testbench also accounts for negative results, such as if Rx is smaller than Ry, but is instructed to perform subtraction, as well as multiplication if either value is negative or both. There was a small number of errors appearing, but as far as concerned, the chances of the error causing system malfunction is trivial, since the test values were large. Hence can be concluded from the testbench that, the ALU is fully functional.

Multiplexer testbench

For the multiplexer testbench, the testing regime is straightforward. The method is to increment all data by one and compare the output from bus to whichever input that corresponds to that specific selector value. After all input is looped through the random values, the selector pin will then increment. As shown in appendix there are no errors.

Register testbench

As well as register, similar to both multiplexer and tick counter's testbenches, register's testbench also utilizes a randomised input approach. Since it functions perfectly in hardware testing, the conventional way of testing would be hard to catch glitches.

Processor testbench

For the main processor. The method to test the extended processor was to have a constantly ticking clock, while incrementing input data with a delay 4 times of clock to ensure output is only checked after 4 ticks, which is from the clock. For non-display operations, the value of Rx is then compared to the value of both Rx and Ry manually operated values to check if it matches on tick 4. For display operation, value in RH is manually compared to value stored inside Rx to check if it matches. Since the processor has a very high complexity level function, it is not possible to test every single combination of inputs (also due to limitations of Modelsim that runs at 32-bits), this was the most efficient way to test as many combinations as possible.

Conclusion

In summary, creating a Verilog processor is a complex but highly rewarding task. It involves careful design, rigorous testing, and a deep understanding of computer architecture and Verilog coding. The flexibility and scalability of Verilog processors make them valuable for various applications, from embedded systems to high-performance computing. However, success in this endeavour relies on continuous testing and quality assurance. Rigorous testing and verification are essential to ensure the processor's reliability and correctness, but it can be said that the Verilog Processor has been perfected.

Appendix A: Verilog modules

```
11  module sign_extend(in, ext);
12      /*
13       * This module sign extends the 9-bit Din to a 16-bit output.
14       */
15
16      // I/O variables:
17      input [8:0] in;
18      output reg [15:0] ext;
19
20      // Implementing sign extender logic:
21      always @(*) begin
22
23          // if LSB of in is 1(negative value), fill ext out with six 1's
24          if (in[8]) begin
25              ext [15:9] <= 7'b 1111111;
26          end
27
28          // else if LSB of in is 0(positive value), fill ext out with six 0's
29          else if (!in[8]) begin
30              ext [15:9] <= 7'b 0000000;
31          end
32
33          // assign rest of ext
34          ext [8:0] <= in;
35      end
36
37  endmodule
```

Figure A1: Verilog module for Sign Extender.

```
 93  module multiplexer(SignExtDin, R0, R1, R2, R3, R4, R5, R6, R7, G, sel, Bus);
 94      /*
 95       * This module takes 10 inputs and places the correct input onto the bus.
 96       */
 97
 98      // I/O variables:
 99      input [15:0] SignExtDin, R0, R1, R2, R3, R4, R5, R6, R7, G;
100      input [3:0] sel;
101      output reg [15:0] Bus;
102
103      // parameters for storing values
104      parameter ZERO =  4'd 0;
105      parameter ONE =   4'd 1;
106      parameter TWO =   4'd 2;
107      parameter THREE = 4'd 3;
108      parameter FOUR =  4'd 4;
109      parameter FIVE =  4'd 5;
110      parameter SIX =   4'd 6;
111      parameter SEVEN = 4'd 7;
112      parameter EIGHT = 4'd 8;
113      parameter NINE =  4'd 9;
114
115      // Implementing multiplexer logic:
116      always @(*) begin
117
118          // case statement to check for output
119          case(sel)
120              ZERO: Bus <= SignExtDin;
121              ONE: Bus <=     R0;
122              TWO: Bus <=     R1;
123              THREE: Bus <=   R2;
124              FOUR: Bus <=    R3;
125              FIVE: Bus <=    R4;
126              SIX: Bus <=     R5;
127              SEVEN: Bus <=   R6;
128              EIGHT: Bus <=   R7;
129              NINE: Bus <=    G;
130          endcase
131      end
132
133  endmodule
```

Figure A2: Verilog Module for Multiplexer.

```
module register_n(r_in, enable, clk, Q, rst);
   /*
    * This module implements registers that will be used in the processor.
    */

   /* To set parameter N during instantiation, you can use:
      register_n #(.N(num_bits)) reg_IR(.....),
      where num_bits is how many bits you want to set N to
      and "..." is your usual input/output signals
    */

   // parameter N to be changed during module instantiation
   parameter N = 16;

   // I/O variables:
   input enable, clk, rst;
   input [N:0] r_in;
   output reg [N:0] Q;

   // Implementing register logic:
   always @(posedge clk) begin

      // if reset off, enable on, store value of r_in input Q
      if(!rst) begin
        if(enable) Q <= r_in;
      end

      // else if reset on, reset Q to store all 0's
      else begin
         Q <= 0;
      end

   end

endmodule
```

Figure A3: Verilog module for Register.

```
module tick_FSM(rst, clk, enable, tick);
   /*
    * This module implements a tick FSM that will be used to
    * control the actions of the control unit
    */

   // I/O variables:
   input rst, clk, enable;
   output reg [3:0] tick;

   // parameters to store values
   parameter STATE1 = 4'b 0001;
   parameter STATE2 = 4'b 0010;
   parameter STATE3 = 4'b 0100;
   parameter STATE4 = 4'b 1000;

   // implementing tick FSM logic:
   always @(posedge clk) begin

      // store state 1 into tick during first run since tick is in state 0000.
      if (tick == 0) tick <= STATE1;

      // if reset 0
      if (!rst) begin

         // if enable 1
         if (enable) begin
            // case check current tick
            case (tick)
               STATE1: tick <= STATE2;
               STATE2: tick <= STATE3;
               STATE3: tick <= STATE4;
               STATE4: tick <= STATE1;
            endcase
         end

         // else if enable 0
         else if (!enable) begin
            // do nothing if enable is off
         end
      end

      // if reset 1
      else if (rst) begin
         // restore tick back to default state
         tick <= 0;
      end
   end
endmodule
```

Figure A4: Verilog module for tick counter.

```
module ALU(input_a, input_b, alu_op, result);
    /*
     * This module implements the arithmetic logic unit of the processor.
     */

    // I/O variables:
    input [15:0] input_a, input_b;
    input [2:0] alu_op;
    wire [31:0] result_temp;
    reg [15:0] temp_a, temp_b;
    output reg [15:0] result;

    // Instantiating lpm_multiplier module
    mult MULT(.dataa(temp_a),.datab(temp_b),.result(result_temp));

    // parameters for storing values
    parameter ZERO =  4'd 0;
    parameter ONE =   4'd 1;
    parameter TWO =   4'd 2;
    parameter THREE = 4'd 3;
    parameter FOUR =  4'd 4;

    // Implementing ALU Logic:
    always @(*) begin

        temp_a = input_a;
        temp_b = input_b;

        // case statement to check operation to perform
        case (alu_op)

            /* alu_op addition*/
            ZERO: begin

                /* performing addition */
                result = input_a + input_b;

                if (input_a[15] == 0 && input_b[15] == 0 && result[15] == 1)
                    result = 16'b 0111111111111111;

                else if (input_a[15] == 1 && input_b[15] == 1 && result[15] == 0)
                    result = 16'b 1000000000000000;
            end

            /* alu_op subtraction*/
            ONE: begin

                /* performing subtraction */
                result = input_a - input_b;

                if (input_a[15] == 0 && input_b[15] == 1 && result[15] == 1)
                    result = input_a[15] == 0 && input_b[15] == 0 ;

                else if (input_a[15] == 1 && input_b[15] == 0 && result[15] == 0)
                    result = 16'b 1000000000000000;
            end

            /* alu_op multiplication */
            TWO: begin

                /* if both input positive */
                if (input_a[15] == 0 && input_b[15] == 0) begin

                    if (result_temp[15:0] > 16'b 0111111111111111)
                        result = 16'b 0111111111111111;
                    else
                        result = input_a * input_b;
                end

                /* if input a negative */
                else if (input_a[15] == 1 && input_b[15] == 0) begin

                    temp_a = ~temp_a + 1'b 1;

                    if (result_temp[15:0] > 16'b 0111111111111111)
                        result = 16'b 1000000000000000;
                    else
                        result = temp_a * temp_b;
                        result = ~result + 1'b 1;
                end

                /* if input a negative */
                else if (input_a[15] == 0 && input_b[15] == 1) begin

                    temp_b = ~temp_b + 1'b 1;

                    if (result_temp[15:0] > (15'b 111111111111111))
                        result = 16'b 1000000000000000;
                    else
                        result = temp_a * temp_b;
                        result = ~result + 1'b 1;
                end

                /* if both input negative */
                else if (input_a[15] == 1 && input_b[15] == 1) begin

                        temp_a = ~temp_a + 1'b 1;
                        temp_b = ~temp_b + 1'b 1;

                        if (result_temp[15:0] > 16'b 0111111111111111)
                            result = 16'b 0111111111111111;
                        else
                            result = temp_a * temp_b;
```

```
237             end
238         end
239
240         /* alu_op shift right */
241         THREE: begin
242
243             /* performing bit shift right */
244             result = input_b >> 1;
245
246             if (input_b[15] == 1)
247                 result[15] = 1'b 1;
248         end
249
250         /* alu_op shift left */
251         FOUR: begin
252             /* performing bit shit left */
253             result = input_b << 1;
254         end
255
256         endcase
257     end
258 endmodule
```

Figure A5: Verilog module for ALU

```
module proc_extension(LEDR, SW, KEY, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5);

    // I/O variables:
    input [1:0] KEY;
    input [9:0] SW;
    output [9:0] LEDR;
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5;

    // wires:
    wire clk, rst;
    wire [8:0] bus, led_disp;
    wire [15:0] display;
    wire [6:0] tick, digit1, digit2, digit3, digit4, digit5;
    reg [15:0] display_final;

    wire [15:0] ten, hundred, thousand, tenk, num4, num3, num2;
    assign ten = 4'd 10;
    assign hundred = 10'd 100;
    assign thousand = 10'd 1000;
    assign tenk = 16'd 10000;

    // wire connections:
    assign clk = ~KEY[0];
    assign rst = SW[9];
    assign bus = SW[8:0];
    assign LEDR[8:0] = led_disp;


    always @(*) begin
        if (display[15] == 1) display_final <= ~display + 1;
        else if (display[15] == 0) display_final <= display;
    end

    bcd_sign bcdsign1(.fullval(display), .in(digit1), .hex(HEX4));

    divide16_v2 divide0(.denom(tenk),
                        .numer(display_final),
                        .quotient(digit1),
                        .remain(num4));

    divide16_v2 divide1(.denom(thousand),
                        .numer(num4),
                        .quotient(digit2),
                        .remain(num3));

    divide16_v2 divide2(.denom(hundred),
                    .numer(num3),
                        .quotient(digit3),
                        .remain(num2));

    divide16_v2 divide3(.denom(ten),
                        .numer(num2),
                        .quotient(digit4),
                        .remain(digit5));

    /* bcd to 7-segment dispplay instances */
    bcd bcd5(.in(tick), .Hex(HEX5));

    bcd bcd3(.in(digit2), .Hex(HEX3));
    bcd bcd2(.in(digit3), .Hex(HEX2));
    bcd bcd1(.in(digit4), .Hex(HEX1));
    bcd bcd0(.in(digit5), .Hex(HEX0));

    /* extended processor instance */
    extended_proc testproc (.clk(clk),
                        .rst(rst),
                        .din(bus),
                        .enable(1),
                        .bus(led_disp),
                        .tick_count(tick),
                        .display(display));

endmodule


module extended_proc(clk, rst, enable, din, bus, tick_count, display, r0, r1, r2, r3, r4, r5, r6, r7);

    // I/O variables:
    input clk, rst, enable;
    input [8:0] din;
    output [3:0] tick_count;
    output [15:0] bus;
    output [15:0] display;

    reg [3:0] bus_control;
    reg [2:0] alu_op;

    // wires:
    wire [15:0] alu_out;

    // register wires
    reg r0_in, r1_in, r2_in, r3_in, r4_in, r5_in, r6_in, r7_in, rG_in, rIR_in, rA_in, rH_in;
    wire [15:0] r0_val, r1_val, r2_val, r3_val, r4_val, r5_val, r6_val, r7_val, rIR_val, rG_val, rA_val, rH_val, sign_ext, bus_wire;

    assign bus = bus_wire;
    assign display = rH_val;

    // output for checking reg value in testbench
    output [15:0]  r0, r1, r2, r3, r4, r5, r6, r7;
    assign r0 = r0_val;
```

```
110     assign r1 = r1_val;
111     assign r2 = r2_val;
112     assign r3 = r3_val;
113     assign r4 = r4_val;
114     assign r5 = r5_val;
115     assign r6 = r6_val;
116     assign r7 = r7_val;
117
118     reg [2:0] Rx, Ry, opcode;
119
120     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
121     /* REGISTERS                                                                                                          */
122     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
123
124     /* General Purpose resister instances
125        instance name: R0, R1, R2, R3, R4, R5, R6, R7
126        general use registers(16-bit).*/
127     register_n #(.N(15)) reg_r0(.r_in(bus_wire), .enable(r0_in), .clk(clk), .Q(r0_val), .rst(rst));
128     register_n #(.N(15)) reg_r1(.r_in(bus_wire), .enable(r1_in), .clk(clk), .Q(r1_val), .rst(rst));
129     register_n #(.N(15)) reg_r2(.r_in(bus_wire), .enable(r2_in), .clk(clk), .Q(r2_val), .rst(rst));
130     register_n #(.N(15)) reg_r3(.r_in(bus_wire), .enable(r3_in), .clk(clk), .Q(r3_val), .rst(rst));
131     register_n #(.N(15)) reg_r4(.r_in(bus_wire), .enable(r4_in), .clk(clk), .Q(r4_val), .rst(rst));
132     register_n #(.N(15)) reg_r5(.r_in(bus_wire), .enable(r5_in), .clk(clk), .Q(r5_val), .rst(rst));
133     register_n #(.N(15)) reg_r6(.r_in(bus_wire), .enable(r6_in), .clk(clk), .Q(r6_val), .rst(rst));
134     register_n #(.N(15)) reg_r7(.r_in(bus_wire), .enable(r7_in), .clk(clk), .Q(r7_val), .rst(rst));
135
136     /* Intruction Register(IR) instance
137        instance name: reg_IR
138        used to store Data in value(9-bit).*/
139     register_n #(.N(8)) reg_IR(.r_in(din), .enable(rIR_in), .clk(clk), .Q(rIR_val), .rst(rst));
140
141     /* Register G instance
142        instance name: reg_G
143        used to store ALU outputvalue(16-bit).*/
144     register_n #(.N(15)) reg_G(.r_in(alu_out), .enable(rG_in), .clk(clk), .Q(rG_val), .rst(rst));
145
146     /* Register A instance
147        instance name: reg_A
148        used to store ALU input_a value(16-bit).*/
149     register_n #(.N(15)) reg_A(.r_in(bus_wire), .enable(rA_in), .clk(clk), .Q(rA_val), .rst(rst));
150
151     /* Register H instance
152        instance name: reg_H
153        used to store HEX5 output value(16-bit).*/
154     register_n #(.N(15)) reg_H(.r_in(bus_wire), .enable(rH_in), .clk(clk), .Q(rH_val), .rst(rst));
155
156     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
157     /*                                                                                                                    */
158     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
160     /* Sign Extender instance
161        instance name: sign_extend
162        used to extend 9-bit DIN to 16bit DIN (2's complement).*/
163     sign_extend sign_extend(.in(din),
164                             .ext(sign_ext));
165
166     /* 10-1 Multiplexer instance
167        instance name: mux
168        used to control which of 10 inputs will be outputted to Bus Wire.*/
169      multiplexer mux( .SignExtDin(sign_ext),
170                       .R0(r0_val),
171                       .R1(r1_val),
172                       .R2(r2_val),
173                       .R3(r3_val),
174                       .R4(r4_val),
175                       .R5(r5_val),
176                       .R6(r6_val),
177                       .R7(r7_val),
178                       .G(rG_val),
179                       .sel(bus_control),
180                       .Bus(bus_wire));
181
182     /* ALU instance
183        instance name: alu
184        used to perform arithmetic operations on input_a & input_b*/
185     ALU alu(.input_a(rA_val),
186             .input_b(bus_wire),
187             .alu_op(alu_op),
188             .result(alu_out));
189
190     /* Tick Counter instance
191        instance name: tick_counter
192        used to control which operations to be performed by control unit*/
193     tick_FSM tick_counter(  .rst(rst),
194                             .clk(clk),
195                             .enable(enable),
196                             .tick(tick_count));
197
198     // states of tick counter (one-hot encoding):
199     parameter STATE1 = 4'b 0001;
200     parameter STATE2 = 4'b 0010;
201     parameter STATE3 = 4'b 0100;
202     parameter STATE4 = 4'b 1000;
203
204     // parameters for storing decimal values:
205     parameter ZERO =  4'd 0;
206     parameter ONE =   4'd 1;
207     parameter TWO =   4'd 2;
208     parameter THREE = 4'd 3;
209     parameter FOUR =  4'd 4;
210     parameter FIVE =  4'd 5;
211     parameter SIX =   4'd 6;
212     parameter SEVEN = 4'd 7;
```

```
213     parameter EIGHT = 4'd 8;
214     parameter NINE =  4'd 9;
215
216
217     // Control unit operations:
218     always @(*) begin
219
220        // Turn on specific control signals based on current tick:
221        case (tick_count)
222
223           default: begin
224              rIR_in <= 1;
225           end
226
227           /* tick 1 */
228           STATE1: begin
229
230              // check for which register to turn off
231              r0_in <= 0;
232              r1_in <= 0;
233              r2_in <= 0;
234              r3_in <= 0;
235              r4_in <= 0;
236              r5_in <= 0;
237              r6_in <= 0;
238              r7_in <= 0;
239
240              rIR_in <= 0;
241              opcode <= rIR_val[8:6];
242              Rx <= rIR_val[5:3];
243              Ry <= rIR_val[2:0];
244
245              // case statement to check ALU operations
246              case (opcode)
247                 ZERO:    alu_op <= SEVEN; // don't care
248                 ONE:     alu_op <= ZERO; // addition
249                 TWO:     alu_op <= ZERO; // addition with immi
250                 THREE:   alu_op <= ONE; // subtraction
251                 FOUR:    alu_op <= TWO; // multiplication
252                 FIVE:    alu_op <= THREE; //srl right 1 bit
253                 SIX:     alu_op <= FOUR; // srl left 1 bit
254                 SEVEN:   alu_op <= SEVEN; // don't care
255              endcase
256
257              /* disp Rx */
258              if (opcode == ZERO) begin
259                 /* Rx value load into bus */
260                 bus_control <= Rx;
261                 rH_in <= 1;
262              end
263
264              /* movi Rx immi */
265              else if (opcode == SEVEN) begin
266                 /* DIN[9:0] value load into bus */
267                 bus_control <= EIGHT;
268                 rH_in <= 0;
269
270                 // check for which register to turn on
271                 if (Rx == ZERO)          r0_in <= 1;
272                 else if (Rx == ONE)      r1_in <= 1;
273                 else if (Rx == TWO)      r2_in <= 1;
274                 else if (Rx == THREE)    r3_in <= 1;
275                 else if (Rx == FOUR)     r4_in <= 1;
276                 else if (Rx == FIVE)     r5_in <= 1;
277                 else if (Rx == SIX)      r6_in <= 1;
278                 else if (Rx == SEVEN)    r7_in <= 1;
279
280              end
281
282              /* Every other operation besides
283                 disp Rx & movi Rx immi     */
284              else if (opcode == ONE | opcode == TWO | opcode == THREE | opcode == FOUR |
285                       opcode == FIVE | opcode == SIX) begin
286                 rH_in <= 0;
287              end
288
289           end
290
291           /* tick 2 */
292           STATE2: begin
293
294              /* disp Rx */
295              if (opcode == ZERO) begin
296                 rH_in <= 0;
297              end
298
299              /* add Rx immi */
300              else if (opcode == TWO) begin
301                 /* DIN[9:0] value load into bus */
302                 bus_control <= EIGHT;
303                 rA_in <= 1;
304              end
305
306              /* srl Rx
307                 sll Rx */
308              else if (opcode == FIVE | opcode == SIX) begin
309                 /* Rx value load into bus */
310                 bus_control <= Rx;
311                 rG_in <= 1;
312
313                 // check for which register to turn on
314                 if (Rx == ZERO)          r0_in <= 1;
315                 else if (Rx == ONE)      r1_in <= 1;
316                 else if (Rx == TWO)      r2_in <= 1;
317                 else if (Rx == THREE)    r3_in <= 1;
```

```
318                  else if (Rx == FOUR)    r4_in <= 1;
319                  else if (Rx == FIVE)    r5_in <= 1;
320                  else if (Rx == SIX)     r6_in <= 1;
321                  else if (Rx == SEVEN)   r7_in <= 1;
322
323               end
324
325               /* movi Rx immi */
326               else if (opcode == SEVEN) begin
327                  r0_in <= 0;
328                  r1_in <= 0;
329                  r2_in <= 0;
330                  r3_in <= 0;
331                  r4_in <= 0;
332                  r5_in <= 0;
333                  r6_in <= 0;
334                  r7_in <= 0;
335               end
336
337               /* add Rx Ry
338                  subt Rx Ry
339                  mult Rx Ry */
340               else if (opcode == ONE | opcode == THREE | opcode == FOUR) begin
341                  /* Rx value load into bus */
342                  bus_control <= Rx;
343                  rA_in <= 1;
344               end
345
346            end
347
348            /* tick 3 */
349            STATE3: begin
350
351               // turn off rA enable pin
352               rA_in <= 0;
353
354               /* disp Rx
355                  movi Rx immi */
356               if (opcode == ZERO | opcode == SEVEN) begin
357                  /* idle */
358               end
359
360               /* add Rx immi */
361               else if (opcode == TWO) begin
362                  /* Rx value load into bus */
363                  bus_control <= Rx;
364                  rG_in <= 1;
365
366               end
367
368               /* srl Rx
369                  sll Rx */
370               else if (opcode == FIVE | opcode == SIX) begin
371                  // bus_control to display output from ALU
372                  bus_control <= NINE;
373                  rG_in <= 0;
374
375                  // check for which register to turn off
376                  r0_in <= 0;
377                  r1_in <= 0;
378                  r2_in <= 0;
379                  r3_in <= 0;
380                  r4_in <= 0;
381                  r5_in <= 0;
382                  r6_in <= 0;
383                  r7_in <= 0;
384               end
385
386               /* add Rx Ry
387                  subt Rx Ry
388                  mult Rx Ry*/
389               else if (opcode == ONE | opcode == THREE | opcode == FOUR) begin
390                  /* Ry value load into bus */
391                  bus_control <= Ry;
392                  // turn on rG and r0 enable pin for next value to be stored during next tick
393                  rG_in <= 1;
394               end
395            end
396
397
398            /* tick 4 */
399            STATE4: begin
400
401               rG_in <= 0;
402               r0_in <= 0;
403
404               rIR_in <= 1;
405
406               /* disp Rx */
407               if (opcode == ZERO) begin
408                  rH_in <= 0;
409               end
410
411               /* movi Rx immi */
412               else if (opcode == SEVEN | opcode == FIVE | opcode == SIX) begin
413                  /* idle */
414               end
```

```
415             /* Every other operation besides
416                disp Rx &  movi Rx immi       */
417             else if (opcode == ONE | opcode == TWO | opcode == THREE | opcode == FOUR) begin
418                // bus_control to display output from ALU
419
420                bus_control <= NINE;
421
422                // check for which register to turn on
423                if (Rx == ZERO)          r0_in <= 1;
424                else if (Rx == ONE)      r1_in <= 1;
425                else if (Rx == TWO)      r2_in <= 1;
426                else if (Rx == THREE)    r3_in <= 1;
427                else if (Rx == FOUR)     r4_in <= 1;
428                else if (Rx == FIVE)     r5_in <= 1;
429                else if (Rx == SIX)      r6_in <= 1;
430                else if (Rx == SEVEN)    r7_in <= 1;
431
432             end
433          end
434
435       endcase
436    end
437 endmodule
```

Figure A6: Verilog module for processor.

Appendix B: Testbench results

```
12  module sign_extend_tb;
13
14      // I/O variables:
15     reg [8:0] in;
16     wire [15:0] out;
17
18     // Instantiate the module:
19     sign_extend test1(in, out);
20
21     initial begin
22        // assign 0 to all inputs
23        in = 0;
24     end
25
26     always begin
27        #10
28        in = in + 1;
29     end
30
31     // create and initialize your testbench statistics
32     integer count, errors;
33
34     initial begin
35        // assign 0 to all variables
36        count = 0;
37        errors = 0;
38     end
39
40     always begin
41        #9
42        if (count == 512) begin
43           #10 /* Display that the code is finished, and state how many errors occured*/;
44           $display("Done with sign_extend test with test count %0d.", count);
45           $display("There were %0d errors.", errors);
46           // Stop the test bench
47           $stop;
48        end
49
50        if (in[8] != out[15] || in[8] != out[14] || in[8] != out[13] || in[8] != out[12] || in[8] != out[11] || in[8] != out[11] ||
51        in[8] != out[10] || in[8] != out[9])
52            errors = errors + 1;
53
54        #1
55        // increment the test case count
56        count = count + 1;
57     end
58
59
60  endmodule
```

Figure B1.1: Verilog testbench module for Sign Extender.

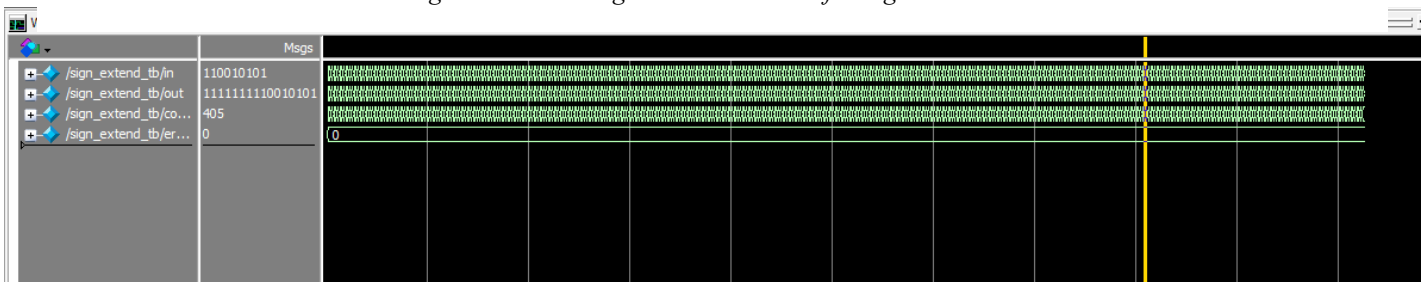

Figure B1.2: Sign Extender waveform.

```
# //
# Loading work.sign_extend_tb
# Loading work.sign_extend
add wave -position end  sim:/sign_extend_tb/in
add wave -position end  sim:/sign_extend_tb/out
add wave -position end  sim:/sign_extend_tb/count
add wave -position end  sim:/sign_extend_tb/errors
# ** Warning: (vsim-WLF-5000) WLF file currently in use: vsim.wlf
#           File in use by: dillo  Hostname: MSI  ProcessID: 21400
#           Attempting to use alternate WLF file "./wlft16teth".
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#           Using alternate file: ./wlft16teth
VSIM 7> run -all
# Done with sign_extend test with test count 512.
# There were 0 errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/sign_extend_tb.v(47)
#    Time: 5139 ns  Iteration: 0  Instance: /sign_extend_tb
# Break in Module sign_extend_tb at C:/intelFPGA_lite/ECE2072/Assignment/sign_extend_tb.v line 47

VSIM 8>
```

Figure B1.3: Modelsim console for Sign Extender.

```
12  module multiplexer_tb;
13
14      // TODO: Implement the logic of your testbench here
15      // Add your code here to create I/O variables
16      reg [15:0] SignExtDin, R0, R1, R2, R3, R4, R5, R6, R7, G;
17      reg [3:0] sel;
18      wire [15:0] bus;
19
20      // Instantiate the module that your are testing
21      multiplexer mux(SignExtDin, R0, R1, R2, R3, R4, R5, R6, R7, G, sel, bus);
22
23      initial begin
24          // set all inputs to 0
25          SignExtDin = 0;
26          R0 = 0;
27          R1 = 0;
28          R2 = 0;
29          R3 = 0;
30          R4 = 0;
31          R5 = 0;
32          R6 = 0;
33          R7 = 0;
34          G = 0;
35          sel = 0;
36      end
37
38      always begin
39          #10
40          // increment input variable
41          SignExtDin = SignExtDin + 1;
42      end
43
44      always begin
45          #20
46          // increment input variable
47          R0 = R0 + 1;
48      end
49
50      always begin
51          #30
52          // increment input variable
53          R1 = R1 + 1;
54      end
55
56      always begin
57          #40
58          // increment input variable
59          R2 = R2 + 1;
60      end
61
62      always begin
63          #50
64          // increment input variable
65          R3 = R3 + 1;
66      end
67
68      always begin
69          #60
70          // increment input variable
71          R4 = R4 + 1;
72      end
73
74      always begin
75          #70
76          // increment input variable
77          R5 = R5 + 1;
78      end
79
80      always begin
81          #80
82          // increment input variable
83          R6 = R6 + 1;
84      end
85
86      always begin
87          #90
88          // increment input variable
89          R7 = R7 + 1;
90      end
91
92      always begin
93          #100
94          // increment input variable
95          G = G + 1;
96      end
97
98      always begin
99          #10
100         // increment input variable
101         sel = sel + 1;
102     end
```

```
103 
104     // create and initialize your testbench statistics
105     integer count, errors;
106 
107     initial begin
108         // Add your code here
109         count = 0;
110         errors = 0;
111     end
112 
113     always begin
114         #9
115         if (count == 524288) begin
116             #10 /* Display that the code is finished, and state how many errors occured*/
117             $display("Done with test.");
118             $display("There were %0d errors.", errors);
119             // Stop the test bench
120             $stop;
121         end
122 
123         if (sel == 0 && (bus != R0))
124             errors = errors + 1;
125         if (sel == 1 && (bus != R1))
126             errors = errors + 1;
127         if (sel == 2 && (bus != R2))
128             errors = errors + 1;
129         if (sel == 3 && (bus != R3))
130             errors = errors + 1;
131         if (sel == 4 && (bus != R4))
132             errors = errors + 1;
133         if (sel == 5 && (bus != R5))
134             errors = errors + 1;
135         if (sel == 6 && (bus != R6))
136             errors = errors + 1;
137         if (sel == 7 && (bus != R7))
138             errors = errors + 1;
139         if (sel == 8 && (bus != SignExtDin))
140             errors = errors + 1;
141         if (sel == 9 && (bus != G))
142             errors = errors + 1;
143 
144         #1
145         // increment the test case count
146         count = count + 1;
147     end
148 
149 
150 endmodule
```

Figure B2.1: Verilog testbench module for multiplexer.

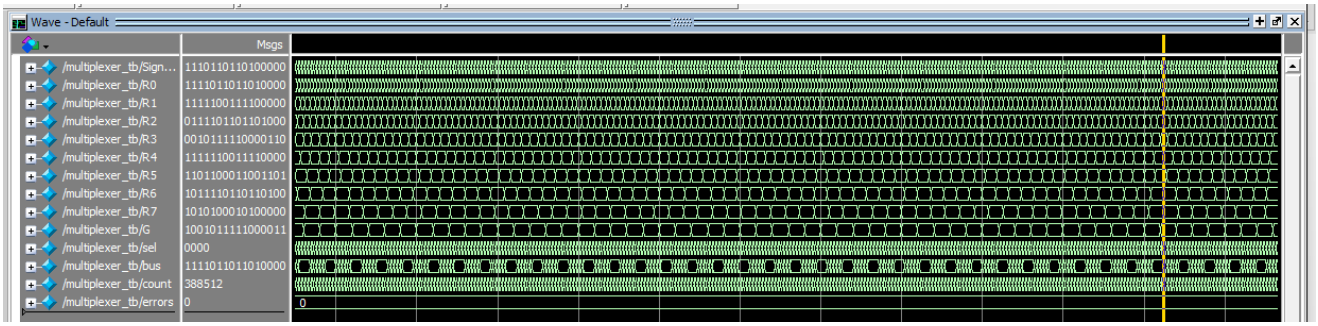

Figure B2.2: Multiplexer waveform.

```
add wave -position end  sim:/multiplexer_tb/R6
add wave -position end  sim:/multiplexer_tb/R7
add wave -position end  sim:/multiplexer_tb/G
add wave -position end  sim:/multiplexer_tb/sel
add wave -position end  sim:/multiplexer_tb/bus
add wave -position end  sim:/multiplexer_tb/count
add wave -position end  sim:/multiplexer_tb/errors
# ** Warning: (vsim-WLF-5000) WLF file currently in use: vsim.wlf
#           File in use by: dillo  Hostname: MSI  ProcessID: 21400
#           Attempting to use alternate WLF file "./wlft6v59ks".
# ** Warning: (vsim-WLF-5001) Could not open WLF file: vsim.wlf
#           Using alternate file: ./wlft6v59ks
VSIM 17> run -all
# Done with test.
# There were 0 errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/multiplexer_tb.v(120)
#    Time: 5242899 ns  Iteration: 0  Instance: /multiplexer_tb
# Break in Module multiplexer_tb at C:/intelFPGA_lite/ECE2072/Assignment/multiplexer_tb.v line 120
```

Figure B2.3: Modelsim console for Multiplexer.

```
12  module ALU_tb;
13
14     // I/O variables
15     reg [15:0] input_a, input_b, test_op3, test_op0, test_op1, test_op2, wire_a, wire_b;
16     wire [31:0] test_mult_hold;
17     reg [2:0] alu_op0,alu_op1,alu_op2,alu_op3,alu_op4;
18     wire [15:0] result_op0,result_op1,result_op2,result_op3,result_op4;
19
20     // Instantiate the module that your are testing
21     ALU test1(input_a, input_b, alu_op0, result_op0);
22     ALU test2(input_a, input_b, alu_op1, result_op1);
23     ALU test3(input_a, input_b, alu_op2, result_op2);
24     ALU test4(input_a, input_b, alu_op3, result_op3);
25     ALU test5(input_a, input_b, alu_op4, result_op4);
26
27     mult MULTIPLICATION(.dataa(wire_a),.datab(wire_b),.result(test_mult_hold));
28
29     initial begin
30        // assign 0 to all inputs
31        input_a = 0;
32        input_b = 0;
33        alu_op0 = 0;
34        alu_op1 = 1;
35        alu_op2 = 2;
36        alu_op3 = 3;
37        alu_op4 = 4;
38     end
39
40     // increament for input_b
41     always begin
42        #10
43        // input_b test until 6 bit since processor input max is 6 bit
44        if (input_b == 16'b 1111111111111111) input_b = 16'b 0111111111111111;
45        else  input_b = input_b + 1;
46     end
47
48     // increament for input_a
49     always begin
50        #327680
51        input_a = input_a + 1;
52     end
53
54     // create and initialize your testbench statistics
55     integer count, add_err, subt_err, mult_err, srl_err, sll_err;
56
57     initial begin
58        /* reseting stat values */
59        count = 0;
60        add_err = 0;
61        subt_err = 0;
62        mult_err = 0;
63        srl_err = 0;
64        sll_err = 0;
65     end
66
67     always @(*) begin
68
69        #9
70        wire_a = input_a;
71        wire_b = input_b;
72
73        if (count==262144) begin
74           #10 /* Display that the code is finished, and state how many errors occured*/;
75           $display("Done with ALU test with test count %0d.", count);
76           $display("There were %0d addition errors.", add_err);
77           $display("There were %0d subtraction errors.", subt_err);
78           $display("There were %0d multiplication errors.", mult_err);
79           $display("There were %0d shift right errors.", srl_err);
80           $display("There were %0d shift left errors.", sll_err);
81           // Stop the test bench
82           $stop;
83        end
84
85        test_op0 = input_a + input_b;
86        if (input_a[15] == 0 && input_b[15] == 0 && test_op0[15]==1)
87           test_op0 = 16'b 0111111111111111;
88        else if (input_a[15] == 1 && input_b[15] == 1 && test_op0[15]==0)
89           test_op0 = 16'b 1000000000000000;
90        if (result_op0 != test_op0)
91            add_err = add_err + 1;
92
93
94
95        test_op1 = input_a - input_b;
96        if (input_a[15] == 0 && input_b[15] == 1 && test_op1[15]==1)
97           test_op1 = input_a[15] == 0 && input_b[15] == 0 ;
98        else if (input_a[15] == 1 && input_b[15] == 0 && test_op1[15]==0)
99           test_op1 = 16'b 1000000000000000;
100       if (result_op1 != test_op1)
101           subt_err = subt_err + 1;
102
103
104       if (input_a[15] == 0 && input_b[15] == 0) begin
105
106          if (test_mult_hold[15:0] > 16'b 0111111111111111)
107             test_op2 = 16'b 0111111111111111;
108          else
```

```
109             test_op2 = input_a * input_b;
110         end
111         else if (input_a[15] == 1 && input_b[15] == 0) begin
112             wire_a = ~wire_a + 1'b 1;
113
114             if (test_mult_hold[15:0] > 16'b 0111111111111111)
115                 test_op2 = 16'b 1000000000000000;
116             else
117                 test_op2 = wire_a * wire_b;
118                 test_op2 = ~test_op2 + 1'b 1;
119         end
120         else if (input_a[15] == 0 && input_b[15] == 1) begin
121             wire_b = ~wire_b + 1'b 1;
122
123             if (test_mult_hold[15:0] > (15'b 111111111111111)) begin
124                 test_op2 = 16'b 1000000000000000;
125             end
126             else begin
127
128                 test_op2 = wire_a * wire_b;
129                 test_op2 = ~test_op2 + 1'b 1;
130             end
131         end
132         else if (input_a[15] == 1 && input_b[15] == 1) begin
133             wire_a = ~wire_a + 1'b 1;
134             wire_b = ~wire_b + 1'b 1;
135
136             if (test_mult_hold[15:0] > 16'b 0111111111111111)
137                 test_op2 = 16'b 0111111111111111;
138             else
139                 test_op2 = wire_a * wire_b;
140
141         end
142         if (result_op2 != test_op2)
143             mult_err = mult_err + 1;
144
145
146         test_op3 = input_b >> 1;
147         if (input_b[15] == 1)
148             test_op3[15] = 1;
149         if (result_op3 != test_op3 )
150             srl_err = srl_err + 1;
151         if (result_op4 != (input_b << 1) )
152             sll_err = sll_err + 1;
153         #1
154         // increment the test case count
155         count = count + 1;
156
157     end
158
```

Figure B3.1: Verilog testbench module for ALU.

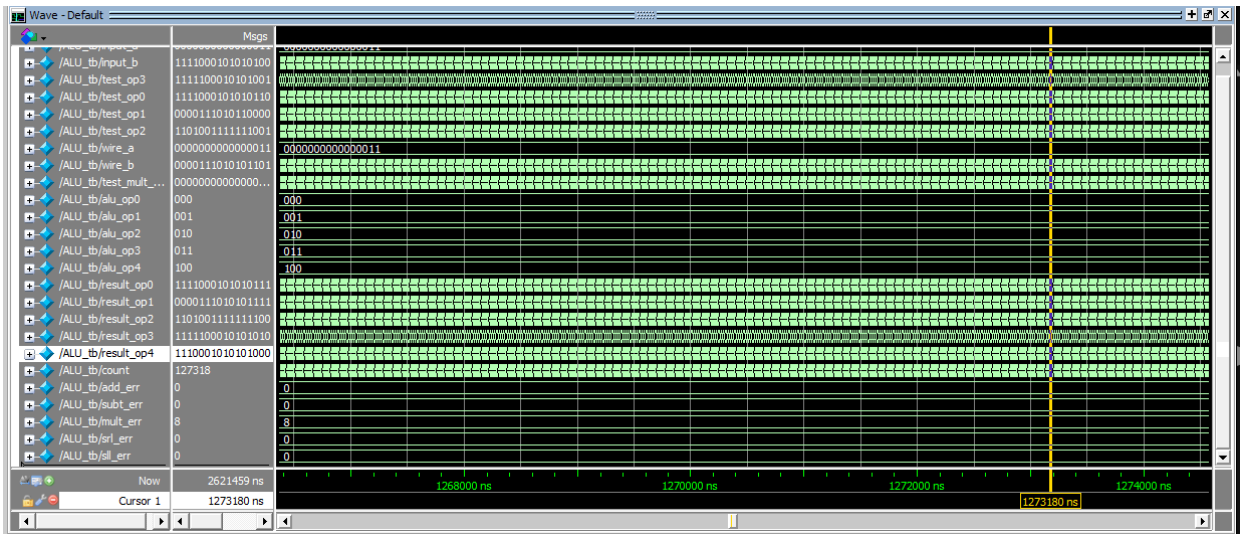

Figure B3.2: ALU waveform.

```
VSIM 28> run -all
# Done with ALU test with test count 262144.
# There were 0 addition errors.
# There were 0 subtraction errors.
# There were 34 multiplication errors.
# There were 0 shift right errors.
# There were 0 shift left errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/ALU_tb.v(82)
#    Time: 2621459 ns  Iteration: 0  Instance: /ALU_tb
# Break in Module ALU_tb at C:/intelFPGA_lite/ECE2072/Assignment/ALU_tb.v line 82
```

Figure B3.3: Modelsim console for ALU testbench.

```
11  module proc_extension_tb;
12      // TODO: Implement the logic of your testbench here
13      reg clk, rst, enable, add_fl, addi_fl, subt_fl, mult_fl, srl_fl, sll_fl, move_fl, disp_fl;
14      reg [15:0] immi;
15      reg [8:0] din;
16      wire [15:0] bus;
17      wire [15:0] display, r0,r1,r2,r3,r4,r5,r6,r7;
18      wire [3:0] tick;
19      reg [15:0] Rx, Ry, sum_add, sum_addi, outputx, sum_subt, sum_mult;
20      wire [15:0] sum_alu, sum_alu_shift;
21      reg [2:0] Rx_temp;
22      reg [2:0] alu_op;
23
24      // parameters for storing decimal values:
25      parameter ZERO =  4'd 0;
26      parameter ONE =   4'd 1;
27      parameter TWO =   4'd 2;
28      parameter THREE = 4'd 3;
29      parameter FOUR =  4'd 4;
30      parameter FIVE =  4'd 5;
31      parameter SIX =   4'd 6;
32      parameter SEVEN = 4'd 7;
33      parameter EIGHT = 4'd 8;
34      parameter NINE =  4'd 9;
35
36
37      // Instantiate the module that your are testing
38      extended_proc testbench_exproc(.clk(clk), .rst(rst), .enable(1), .din(din),
39      .bus(bus), .tick_count(tick), .display(display), .r0(r0), .r1(r1),
40      .r2(r2), .r3(r3), .r4(r4), .r5(r5), .r6(r6), .r7(r7));
41
42      ALU alu1(.input_a(Rx), .input_b(Ry), .alu_op(alu_op), .result(sum_alu));
43      ALU ALU_shift(.input_a(), .input_b(Rx), .alu_op(alu_op), .result(sum_alu_shift));
44
45      initial begin
46          clk = 1;
47          rst = 1;
48          enable = 1;
49  //      flag = 0;
50          din = 9'b 000000000;
51      end
52
53      initial begin
54          #80
55          rst = 0;
56          din = 9'b 111001000;
57      end
58
59  // initial begin
60  //     #150
61  ////      flag = 1;
62  // end
63
64      always begin
65          #10
66          clk = clk + 1'b 1;
67      end
68
69      always begin
70          #80
71          case (din[8:6])
72              ZERO:   alu_op <= SEVEN; // don't care
73              ONE:    alu_op <= ZERO; // addition
74              TWO:    alu_op <= ZERO; // addition with immi
75              THREE:  alu_op <= ONE; // subtraction
76              FOUR:   alu_op <= TWO; // multiplication
77              FIVE:   alu_op <= THREE; //srl right 1 bit
78              SIX:    alu_op <= FOUR; // srl left 1 bit
79              SEVEN:  alu_op <= SEVEN; // don't care
80          endcase
81
82          // increament din by 1 every 4 clocks
83          din = din + 1;
84
85      end
86
87      // initialising stats
88      integer count, error, add_error, addi_error, subt_error, mult_error, srl_error, sll_error, move_error, disp_error;
89
90      initial begin
91          // setting every stat to 0
92          count = 0;
93          error = 0;
94          add_error = 0;
95          addi_error = 0;
96          subt_error = 0;
97          mult_error = 0;
98          srl_error = 0;
99          sll_error = 0;
100         move_error = 0;
101         disp_error = 0;
102     end
103
104     always begin
105         #18
106         if (count == 5000) begin
107             #10
108             $display("Done with processor test with test count %0d.", count);
109             $display("There were %0d errors.", error);
110             $stop;
111         end
112
113         if (tick == 4'b 0001) begin
114
```

```
115             sum_mult = sum_alu;
116             case (Rx_temp)
117                 3'b000: outputx = r0;
118                 3'b001: outputx = r1;
119                 3'b010: outputx = r2;
120                 3'b011: outputx = r3;
121                 3'b100: outputx = r4;
122                 3'b101: outputx = r5;
123                 3'b110: outputx = r6;
124                 3'b111: outputx = r7;
125             endcase
126
127             if ((sum_add != outputx) && add_fl == 1) begin
128                 add_error = add_error + 1;
129             end
130             if ((sum_addi != outputx) && addi_fl == 1) begin
131                 addi_error = addi_error + 1;
132             end
133             if ((sum_subt != outputx) && subt_fl == 1) begin
134                 subt_error = subt_error + 1;
135             end
136             if ((sum_mult != outputx) && mult_fl == 1) begin
137                 mult_error = mult_error + 1;
138             end
139             if ((sum_alu_shift != outputx) && srl_fl == 1) begin
140                 srl_error = srl_error + 1;
141             end
142             if ((sum_alu_shift != outputx) && sll_fl == 1) begin
143                 sll_error = sll_error + 1;
144             end
145             if ((immi != outputx) && move_fl == 1) begin
146                 move_error = move_error + 1;
147             end
148             if ((immi != outputx) && disp_fl == 1) begin
149                 disp_error = disp_error + 1;
150             end
151
152             // set all operation  to 0;
153             add_fl = 0;
154             addi_fl = 0;
155             subt_fl = 0;
156             mult_fl = 0;
157             srl_fl = 0;
158             sll_fl = 0;
159             move_fl = 0;
160             disp_fl = 0;
161
162         end
163
164         // store Rx value
165         case (din[5:3])
166             3'b000: Rx = r0;
167             3'b001: Rx = r1;
168             3'b010: Rx = r2;
169             3'b011: Rx = r3;
170             3'b100: Rx = r4;
171             3'b101: Rx = r5;
172             3'b110: Rx = r6;
173             3'b111: Rx = r7;
174         endcase
175
176         // store Ry value
177         case (din[2:0])
178             3'b000: Ry = r0;
179             3'b001: Ry = r1;
180             3'b010: Ry = r2;
181             3'b011: Ry = r3;
182             3'b100: Ry = r4;
183             3'b101: Ry = r5;
184             3'b110: Ry = r6;
185             3'b111: Ry = r7;
186         endcase
187
188         case (din[8:6])
189             // 001: Add rx
190             3'b001: begin
191                 if (tick == 4'b 0001) begin
192
193                     Rx_temp = din[5:3];
194
195                 end
196                 if (tick == 4'b 1000)begin
197                     sum_add = Rx + Ry;
198
199                     if (Rx[15] == 0 && Ry[15] == 0 && sum_add[15] == 1)
200                         sum_add = 16'b 0111111111111111;
201
202                     else if (Rx[15] == 1 && Ry[15] == 1 && sum_add[15] == 0)
203                         sum_add = 16'b 1000000000000000;
204
205                     add_fl = 1;
206
207                 end
208             end
209             3'b010: begin
210                 if (tick == 4'b 0001) begin
211                     Rx_temp = din[5:3];
212                     case (din[5:3])
213                         3'b000: Rx = r0;
214                         3'b001: Rx = r1;
215                         3'b010: Rx = r2;
216                         3'b011: Rx = r3;
217                         3'b100: Rx = r4;
218                         3'b101: Rx = r5;
```

```
                    3'b110: Rx = r6;
                    3'b111: Rx = r7;
                endcase
            end
            if (tick == 4'b 0010) begin
                immi = bus;
            end

            if (tick == 4'b 1000)begin
                sum_addi = Rx + immi;

                if (Rx[15] == 0 && immi[15] == 0 && sum_addi[15] == 1)
                    sum_addi = 16'b 0111111111111111;

                else if (Rx[15] == 1 && immi[15] == 1 && sum_addi[15] == 0)
                    sum_addi = 16'b 1000000000000000;

                addi_fl = 1;
            end

        end
        3'b011: begin
            if (tick == 4'b0001)begin
                Rx_temp = din[5:3];


            end
            if (tick == 4'b1000) begin
                sum_subt = Rx - Ry;

                if (Rx[15] == 0 && Ry[15] == 1 && sum_subt[15] == 1)
                    sum_subt = Rx[15] == 0 && Ry[15] == 0 ;
                else if (Rx[15] == 1 && Ry[15] == 0 && sum_subt[15] == 0)
                    sum_subt = 16'b 1000000000000000;

                subt_fl = 1;
            end
        end
        3'b100: begin
            if (tick == 4'b0001)begin
                Rx_temp = din[5:3];


            end
            if (tick == 4'b1000) begin


                mult_fl = 1;
            end
        end
        3'b101: begin
            if (tick == 4'b0001) begin
                Rx_temp = din[5:3];

            end
            if (tick == 4'b1000)begin
                srl_fl = 1;
            end

        end
        3'b110: begin
            if (tick == 4'b0001) begin

            end
            if (tick == 4'b1000)begin
                sll_fl = 1;
            end

        end
        3'b111: begin
            if (tick == 4'b0001) begin
                Rx_temp = din[5:3];
    //
            end
            if (tick == 4'b0010) begin
                immi = bus;
            end
            if (tick == 4'b1000)begin
                move_fl = 1;
            end

        end
        3'b000: begin
            if (tick == 4'b0001) begin
                Rx_temp = din[5:3];
    //
            end

            if (tick == 4'b1000)begin
                immi = display;
                disp_fl = 1;
            end

        end


    endcase

    #2
    count = count + 1;
  end

endmodule
```

Figure B4.1: Verilog testbench module for processor.

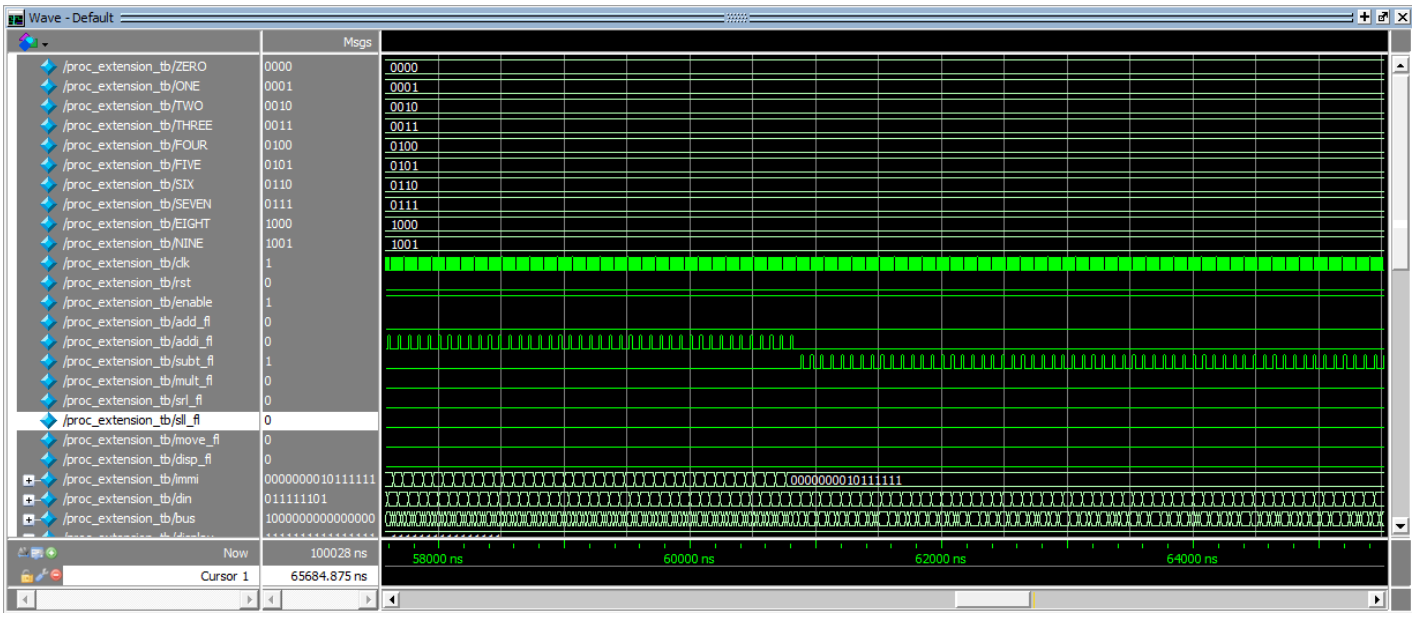

Figure B4.2: Processor testbench waveform.

```
VSIM 58> run -all
# Done with processor test with test count 5000.
# There were 0 errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/proc_extension_tb.v(110)
#    Time: 100028 ns  Iteration: 0  Instance: /proc_extension_tb
# Break in Module proc_extension_tb at C:/intelFPGA_lite/ECE2072/Assignment/proc_extension_tb.v line 110

VSIM 59>
```

Figure B4.3: Modelsim console for processor testbench.

```
12  module register_tb;
13
14      // I/O variables:
15     reg [15:0] in;
16     wire [15:0] out;
17     reg enable, clk, rst;
18
19     // Instantiate the module:
20     register_n test1(.r_in(in), .enable(enable), .clk(clk), .Q(out), .rst(rst));
21
22     initial begin
23        // assign 0 to all inputs
24        in <= 0;
25        enable <= 0;
26        clk <= 0;
27        rst <= 0;
28     end
29
30     always begin
31        #10
32        clk = clk + 1;
33     end
34
35     always begin
36        #20
37        in = in + 1;
38     end
39
40     always begin
41        #200
42        enable = enable + 1;
43     end
44
45     always begin
46        #400
47        rst = rst + 1;
48     end
49
50     // create and initialize your testbench statistics
51     integer count, errors;
52
53     initial begin
54        // assign 0 to all variables
55        count = 0;
56        errors = 0;
57     end
58
59    always begin
60       #9
61       if (count == 800) begin
62          #10 /* Display that the code is finished, and state how many errors occured*/;
63          $display("Done with register test with test count %0d.", count);
64          $display("There were %0d errors.", errors);
65          // Stop the test bench
66          $stop;
67       end
68
69       if (clk && enable && !rst && (out != in))
70          errors = errors + 1;
71
72       if (clk && rst && (out != 0))
73          errors = errors + 1;
74
75
76       #1
77       // increment the test case count
78       count = count + 1;
79    end
80
81
8
```

Figure B5.1: Verilog testbench module for register.

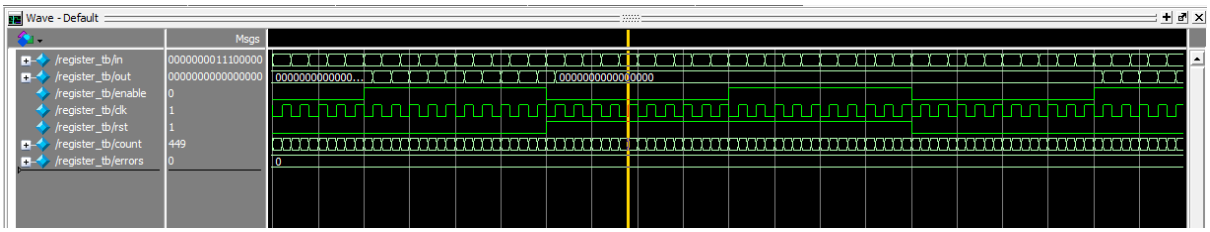

Figure B5.2: Register waveform.

```
VSIM 10> run -all
# Done with register test with test count 800.
# There were 0 errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/register_tb.v(66)
#    Time: 8019 ns  Iteration: 0  Instance: /register_tb
# Break in Module register_tb at C:/intelFPGA_lite/ECE2072/Assignment/register_tb.v line 66
```

Figure B5.3: Modelsim console for register.

```
12  module tick_FSM_tb;
13
14      // TODO: Implement the logic of your testbench here
15      // Add your code here to create I/O variables
16    reg rst, clk, enable;
17    wire [3:0] tick;
18
19    // Instantiate the module that your are testing
20    tick_FSM tick_counter_test(rst, clk, enable, tick);
21
22    initial begin
23       // Add your code here
24       rst = 0;
25       clk = 0;
26       enable = 0;
27    end
28
29    always begin
30       #10
31       // increment your input variables
32       clk = clk + 1;
33    end
34
35    always begin
36       #160
37       // increment your input variables
38       enable = enable + 1;
39    end
40
41    always begin
42       #400
43       // increment your input variables
44       rst = rst + 1;
45    end
46
47    // create and initialize your testbench statistics
48    integer count, errors;
49
50    initial begin
51       // Add your code here
52       count = 0;
53       errors = 0;
54    end
55
56    always begin
57       #9
58       if (count == 500) begin
59          #10 /* Display that the code is finished, and state how many errors occured*/;
60          $display("Done with tick_FSM test with count %0d.", count);
61          $display("There were %0d errors.", errors);
62          // Stop the test bench
63          $stop;
64       end
65
66       if (rst && tick != 0 && clk) begin
67          errors = errors + 1;
68       end
69
70       #1
71       // increment the test case count
72       count = count + 1;
73    end
74
75
76  endmodule
```

Figure B6.1: Verilog testbench module for tick counter.

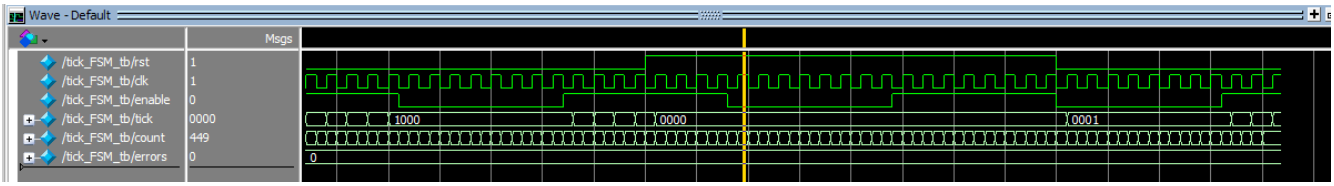

Figure B6.2: Tick counter waveform

```
VSIM 9> run -all
# Done with tick_FSM test with count 500.
# There were 0 errors.
# ** Note: $stop    : C:/intelFPGA_lite/ECE2072/Assignment/tick_FSM_tb.v(63)
#    Time: 5019 ns  Iteration: 0  Instance: /tick_FSM_tb
# Break in Module tick_FSM_tb at C:/intelFPGA_lite/ECE2072/Assignment/tick_FSM_tb.v line 63
```

Figure B6.3: Modelsim console for tick counter.